Dynamic Analysis - Fuzzing IoT Devices

We acknowledge funding provided by Microelectronic Commons and the support of OUSW(R&E).

Invalid State Detection

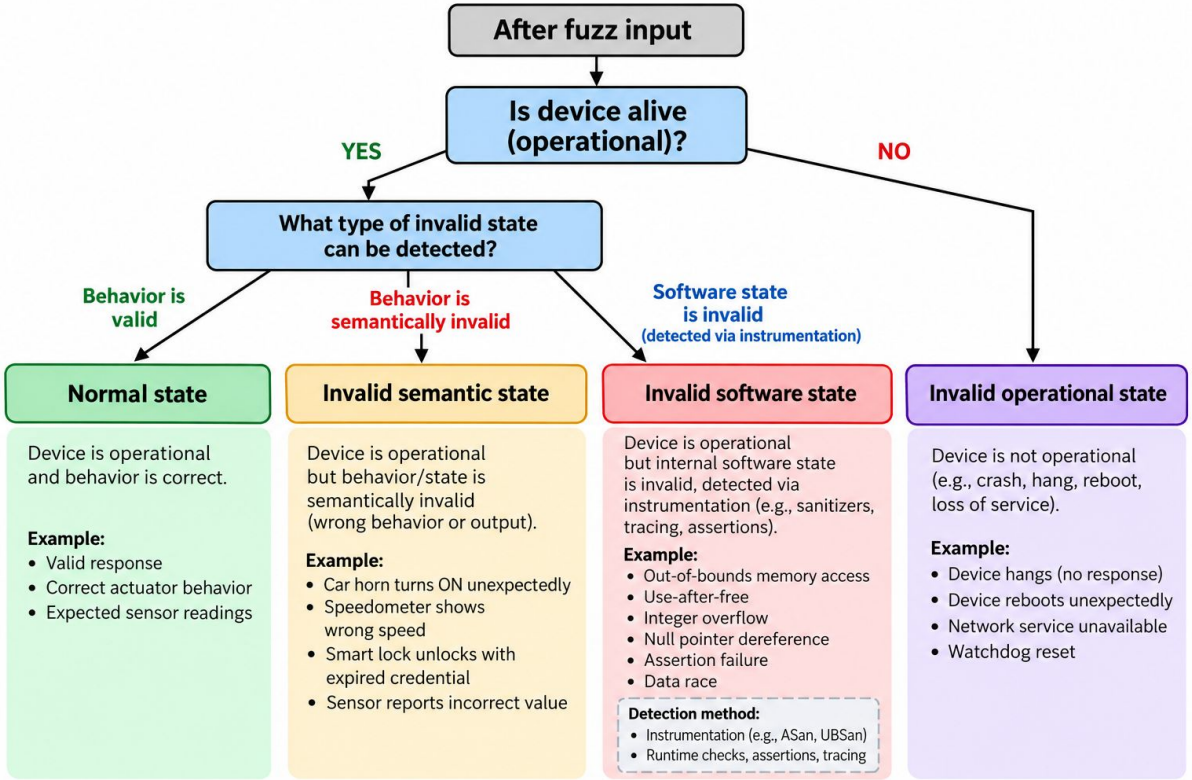
Invalid State Detection

- Invalid semantic state
- Invalid operational state
- Invalid software state

Invalid State Detection

- Invalid semantic state:
  - Device is operational, but the state is semantically invalid.
- Invalid operational state:
  - Device is not operational and it is not expected.
    - Expected v/s Unexpected
- Invalid software state:
  - The software had reached an unexpected state.

Invalid State Detection



Invalid Semantic State Detection

Pros:

+ Detects deeper, safety-relevant bugs
+ Finds failures without crashes
+ Require only semantic model

Cons:

– Requires knowing what “correct behavior” means
– Hard to generalize across devices

Invalid Operational State Detection

Pros:

+ Easy, inexpensive, highly automatable

Cons:

− Detects mostly crashes, hangs, and reboots

− Misses “alive but wrong” behavior

Invalid Software State Detection

Pros:

+ Detects invisible invalid states

+ Aids in easy fixing

Cons:

– Extremely intrusive

- Slow, often require source code

Invalid Operational State Detection

- Heartbeat / liveness — IoTFuzzer (NDSS ’18): Send a known-good status request; missing heartbeat → possible crash/hang.

- Timeout monitoring — Snipuzz (CCS ’21): Repeated missing responses → replay input and classify persistent timeout as a crash.

- Connection monitoring — IoTFuzzer (NDSS ’18): Unexpected TCP disconnection → device/service likely crashed.

Invalid Operational State Detection

- Reboot monitoring — SweynTooth (ATC ’20): Detect loss of BLE operation followed by device restart; demonstrated on Fitbit Inspire, Eve Energy, and August Smart Lock.

- CPU exception monitoring — µEmu (USENIX Security ’21): HardFault or invalid memory access → crash; execution timeout → hang.

- Debug-interface monitoring — GDBFuzz (ISSTA ’23): JTAG/SWD/GDB observes halted or crashed execution and records the triggering input.

- UART / log monitoring — Inter-CESTI (SSTIC ’20): Parse serial output for panic, HardFault, and register/stack dumps.

- Physical side-channel monitoring — Side-Channel Aware Fuzzing (ESORICS ’19): Use externally measured power traces to infer abnormal execution when software feedback is unavailable.

Invalid Software State Detection

Detect invalid states in software. Why?

Bug Manifestations

- How bugs affect program behavior?
  - If we have exhaustive test cases:
    - Actual output != Expected output.
  - In the absence of test cases, i.e., Fuzzing:
    - Memory errors: Program Crashes (SIGSEGV) => Access/Execute invalid memory.
    - There could be bugs which do not result in SIGSEGV.

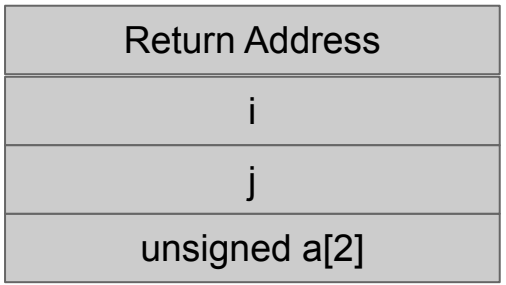
Silent Bugs

```
int main() {
  unsigned i, j, a[2];
  scanf("%u %u", &i, &j);
  a[i] = j;
  ...
  return 0;
}
```

Here, if i == 2 (off by one error), the program may not crash?

- Why?

Runtime Stack

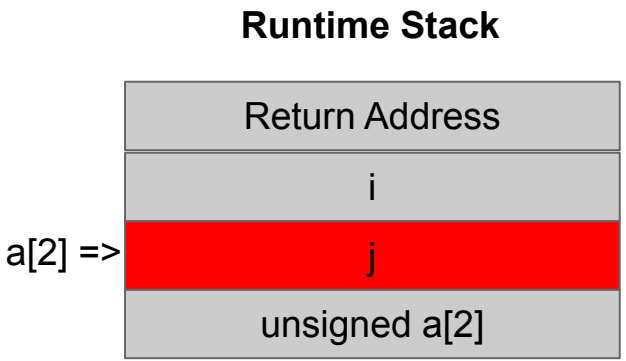


Silent Bugs

```
int main() {
  unsigned i, j, a[2];
  scanf("%u %u", &i, &j);
  a[i] = j;
  ...
  return 0;
}
```

Here, if i == 2 (off by one error), the program may not crash?

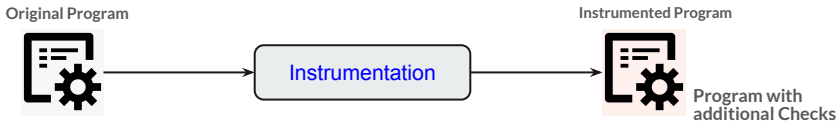
- Why?



Sanitizers

- Change the program such that ***we detect bugs when they occur*** instead of waiting for the bugs to result in crash.
- Mechanism: Instrument the program by adding additional checks for detecting bugs.

Sanitizers: Overview



Original Program

```
int main() {
  unsigned i, j, a[2];
  scanf("%u %u", &i, &j);
  a[i] = j;
  ...
  return 0;
}
```

Array out-of-bounds Sanitizer

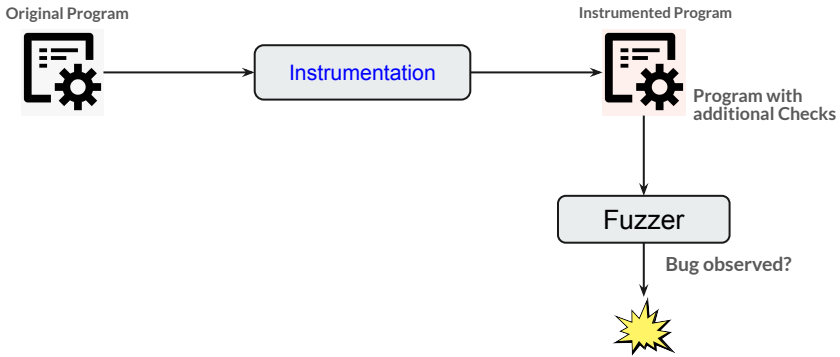
```
int main() {
  unsigned i, j, a[2];
  scanf("%u %u", &i, &j);
  if (i < 2) {
    a[i] = j;
  } else { CRASH}
  ...
  return 0;
}
```

Instrumented Program

Real world Sanitizers

- Usually bug specific. Examples:
  - MemorySanitizer: Detects *uninitialized reads*.
  - AddressSanitizer: Detects *invalid memory accesses*.
- **General instrumentation idea**: At all instructions in the program where the bug can occur, add a check to detect the bug.
  - AddressSanitizer: Detects Invalid Memory Accesses.
    - Invalid Memory access can occur at load and store instructions.
      - **Instrument every load and store to check if the used address is invalid (i.e., does not belong to a program object).**

Sanitizers: Usage



Original Program

```
int main() {
  unsigned i, j, a[2];
  scanf("%u %u", &i, &j);
  a[i] = j;
  ...
  return 0;
}
```

Array out-of-bounds Sanitizer

```
int main() {
  unsigned i, j, a[2];
  scanf("%u %u", &i, &j);
  if (i < 2) {
    a[i] = j;
  } else { CRASH}
  ...
  return 0;
}
```

Instrumented Program



Why can’t we always use sanitizers?

They detect bugs at runtime => Why can’t we just use sanitizers and not worry about bugs, as they will never lead to vulnerabilities.

Sanitizers Implementation

- Sanitizers need to maintain lot of additional state to check for the possibility of bugs.
  - AddressSanitizer: Detects Invalid Memory Accesses:
    - Need to **maintain metadata** regarding which memory (i.e., address) is valid v/s invalid.
    - Tricky: Handling dynamic memory allocation.

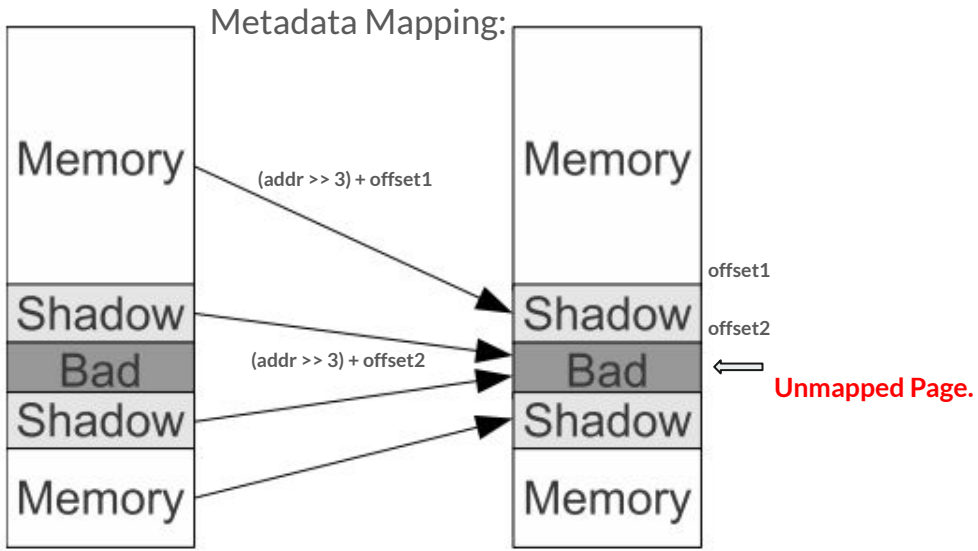
Popular research direction: Smart and efficient way to maintain metadata.

AddressSanitizer (ASan)

- Metadata (or shadow memory):
  - One eighth of the virtual memory will be used to maintain metadata:
    - One bit of metadata for each byte of application memory.
    - Bit is zero: The corresponding address is valid else invalid.
- Accessing metadata for a given address (Addr):
  - Direct Mapping:

```
// Checking 8-byte access
MetadataAddr = (Addr >> 3) + Offset;
if (*MetadataAddr != 0)
  ReportAndCrash(Addr);
```

ASan: Mapping



ASan: Usage

a.c

```
void foo(T *a) {
*a = 0x1234;
}
```



clang **-fsanitize=address** a.c -c -DT=long



```
push %rax
mov %rdi,%rax
shr $0x3,%rax
mov $0x100000000000,%rcx
or %rax,%rcx
cmpb $0x0,(%rcx) # Compare Shadow with 0
jne 23 <foo+0x23> # To Error
movq $0x1234,(%rdi) # Original store
pop %rax
retq
callq __asan_report_store8 # Error
```

ASAN on Binary Firmware

